22/09/2026

Information Retrieval: Using Apache Solr

Indexing, Retrieval Models, Query Processing & Evaluation

Submitted By: Amit Kumar

Information Retrieval: Using Apache Solr

Indexing, Retrieval Models, Query Processing & Evaluation

1. Objective

To gain hands-on experience in indexing, retrieval models, query processing, and evaluation of search systems using Apache Solr as the industry-standard IR platform.

2. Tool Used

Apache Solr version 10.0.0, run in standalone SolrCloud (embedded ZooKeeper) mode on Windows 10, with Java 27 (OpenJDK, HotSpot 64-Bit Server VM) as the runtime.

3. Dataset and Index Construction

A collection of 55 original short technology-news style documents was created, covering Python, Java, Machine Learning, Cloud Computing, Web Development, Databases, Cybersecurity, Mobile Development, Blockchain, and DevOps. Topic overlap between documents was deliberately introduced (e.g. a document mentioning both Python and Java) so that Boolean and ranked-retrieval queries return meaningful, explainable results rather than trivial all-or-nothing matches.

Each document has the following fields:

- id — unique document identifier (e.g. doc001)
- title — article headline
- body — 2–3 sentence article content (main searchable text)
- category — topic category
- author — author name
- tags — multi-valued keyword tags

Full dataset: documents.json (55 records). Field-level description: dataset_description.md (both submitted alongside this report).

4. Indexing Pipeline

The indexing pipeline followed these steps:

- 1. Installed Apache Solr 10.0.0 on Windows and started it in SolrCloud (embedded ZooKeeper) mode on port 8983: bin\solr.cmd start -p 8983
- 2. Created a core/collection named 'ir_assignment': bin\solr.cmd create -c ir_assignment

- 3. Because Solr 10 stores schema configuration in ZooKeeper rather than as a static local XML file, the 5 custom fields (title, body, category, author, tags) were added using Solr's Schema API rather than hand-editing managed-schema.
- 4. text_general (used for title/body) tokenizes on whitespace/punctuation, lowercases tokens, and removes English stopwords, building the inverted index postings for full-text search.
- 5. Bulk-indexed all 55 documents in one request via the /update/json/docs endpoint using documents.json.
- 6. Verified the index by running q=*:* and confirming numFound = 55.

Schema fields added (via Schema API):

```
POST http://localhost:8983/solr/ir_assignment/schema
{
  "add-field": [
    {"name":"title","type":"text_general","indexed":true,"stored":true},
    {"name":"body","type":"text_general","indexed":true,"stored":true},
    {"name":"category","type":"string","indexed":true,"stored":true},
    {"name":"author","type":"string","indexed":true,"stored":true},

{"name":"tags","type":"string","indexed":true,"stored":true,"multiValued":tr
ue}
  ]
}
```

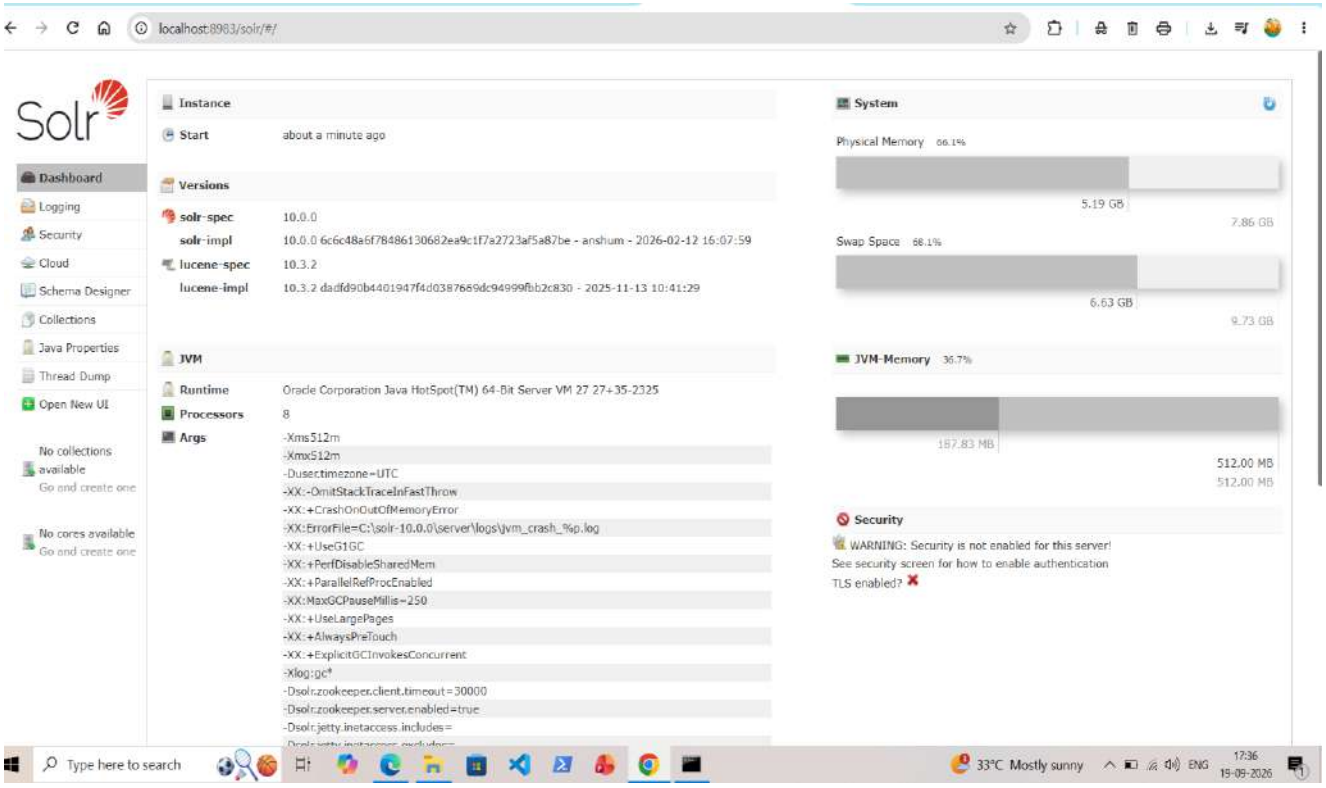


Fig 1 — Solr 10.0.0 Admin Dashboard confirming the server is running.

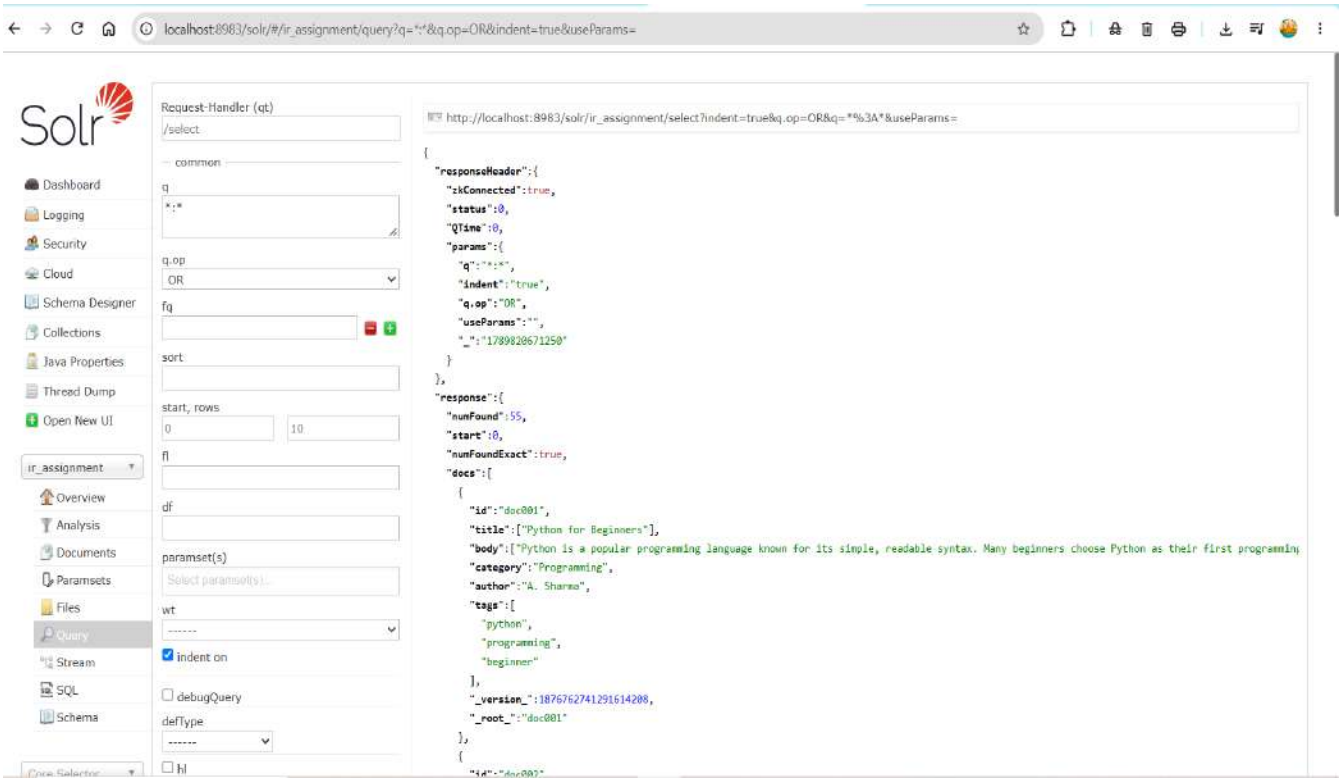


Fig 2 — Verification query q=:* returning numFound: 55, confirming all documents were indexed successfully with all fields (title, body, category, author, tags) populated.*

5. Boolean Queries

Four Boolean queries were executed against the body field of the indexed collection.

I. Term1 AND Term2

```
q = body:python AND body:machine
```

Result: numFound = 1. Only doc004 ("Machine Learning with Python") contains both terms — the strict intersection behaviour of AND.

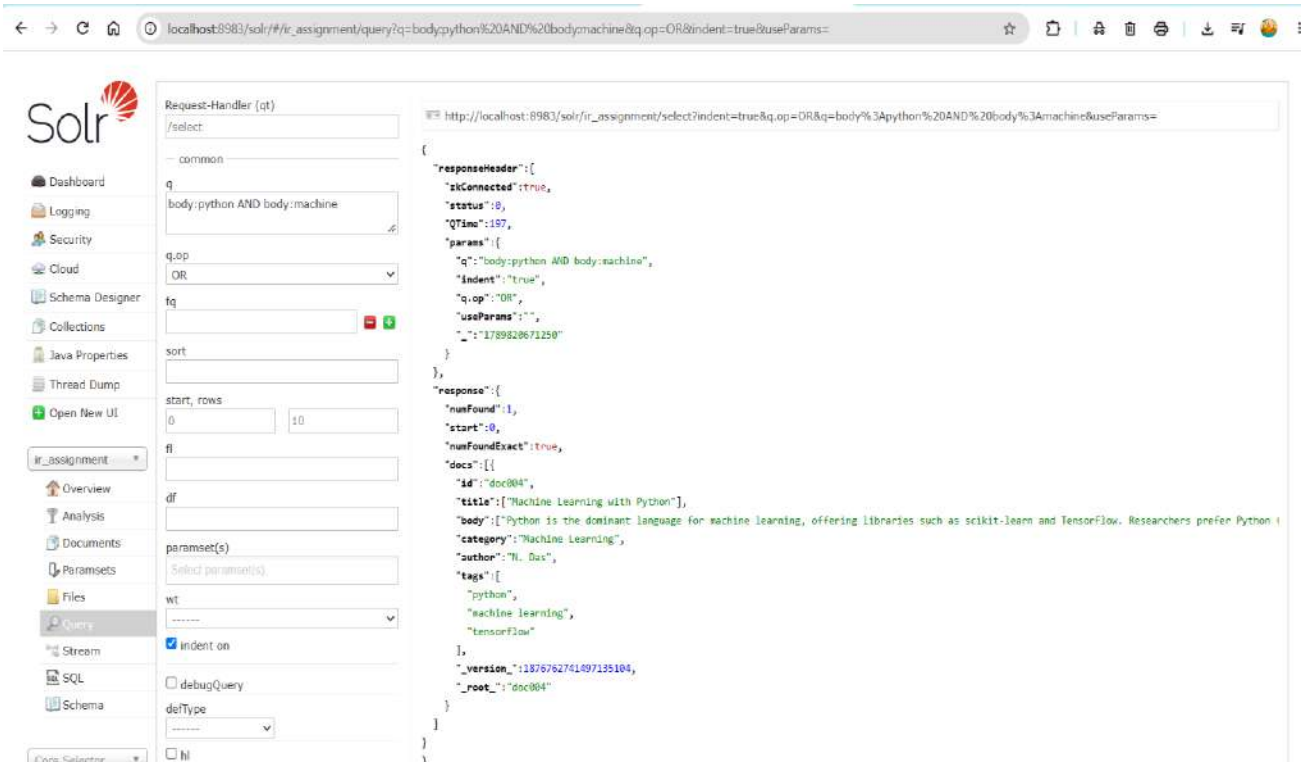


Fig 3 — AND query result: 1 match (doc004).

II. Term1 OR Term2

```
q = body:python OR body:java
```

Result: numFound = 17 — the union of all 10 Python documents and all 7 Java documents, with doc007 ("Python vs Java Performance", which mentions both terms) counted only once, as expected for a union/OR operation.

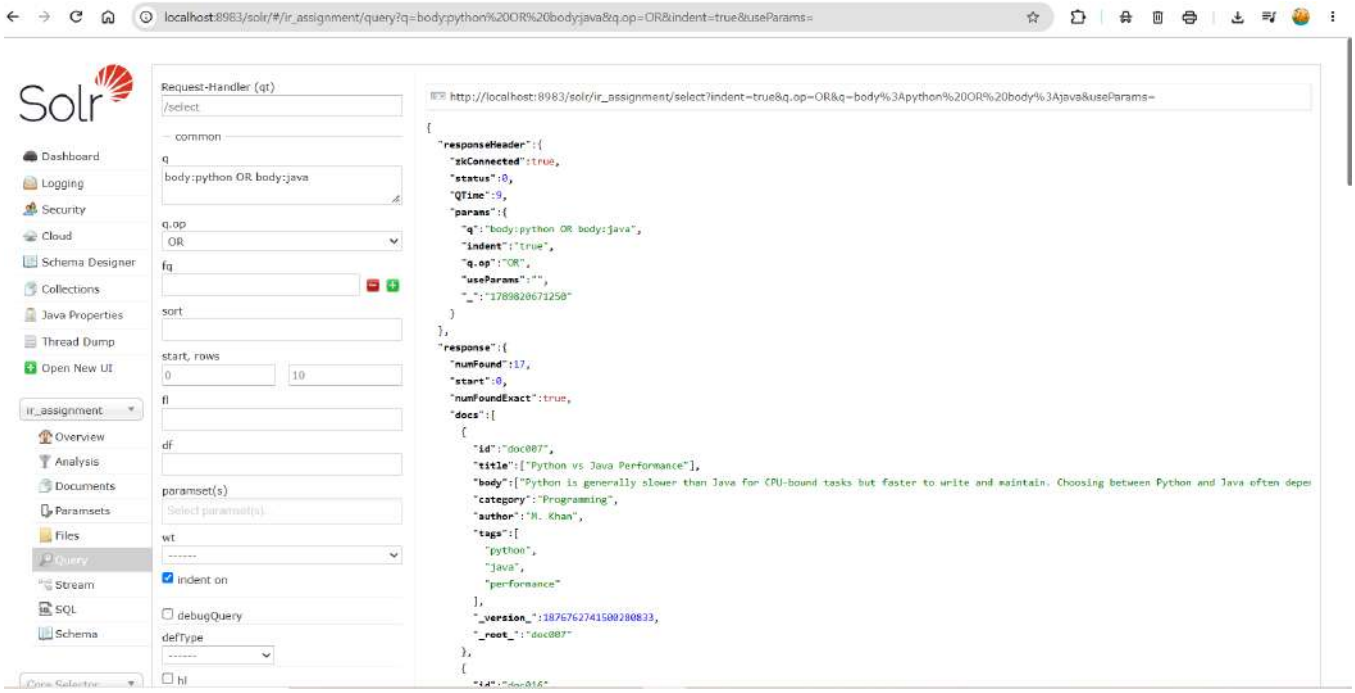


Fig 4 — OR query result: 17 matches.

III. Term1 AND NOT Term2

```
q = body:python AND NOT body:java
```

Result: numFound = 9 — all Python documents except doc007, which was correctly excluded because it also mentions “java”.

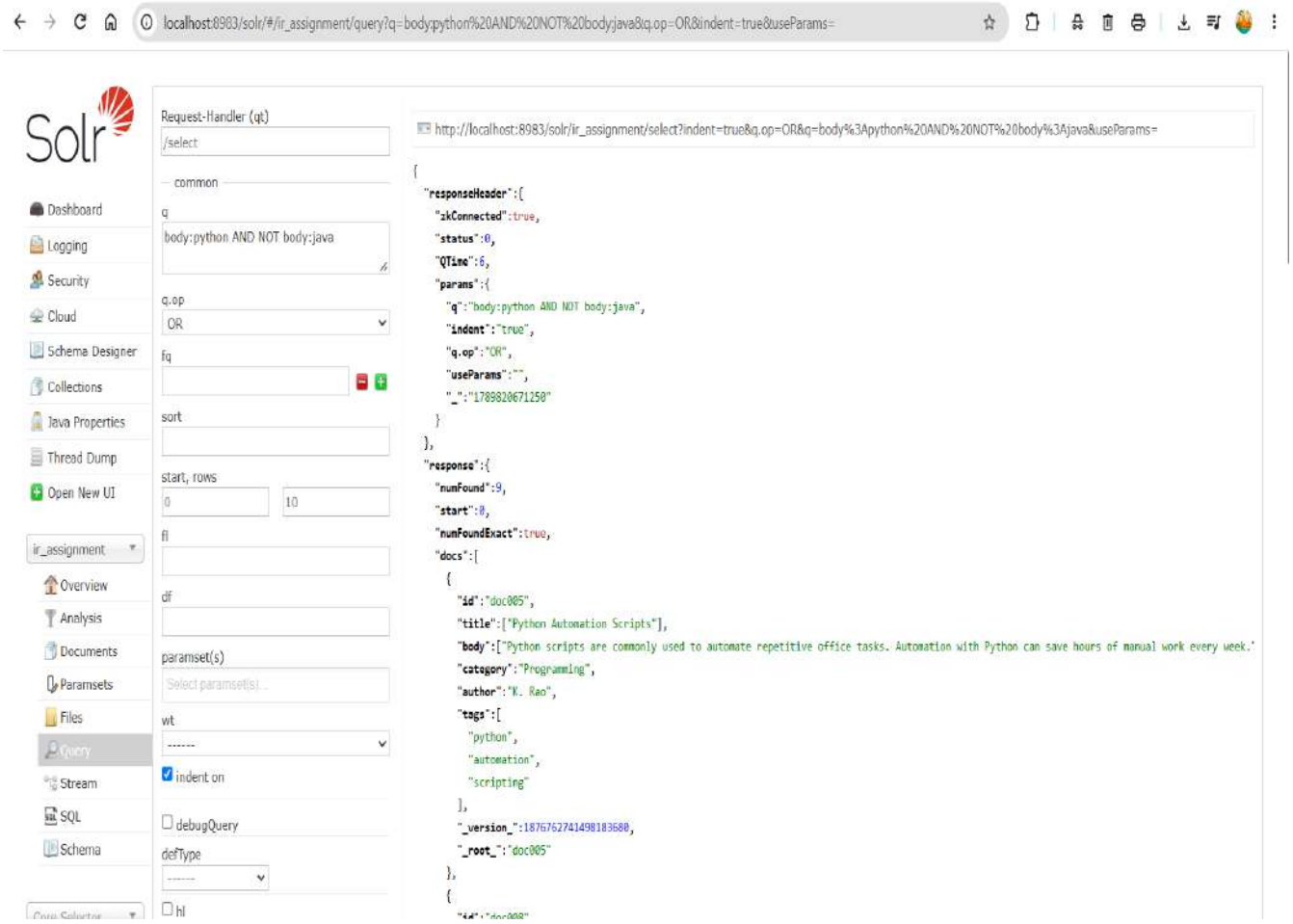


Fig 5 — AND NOT query result: 9 matches (doc007 excluded).

IV. Nested Boolean Query

```
q = (body:python OR body:java) AND (body:web OR body:cloud) AND
NOT body:mobile
```

Result: numFound = 2 — doc003 ("Python Web Frameworks") and doc013 ("Spring Framework for Java"), the only documents about Python/Java that also mention web or cloud, while documents about mobile development were correctly excluded by the AND NOT clause.

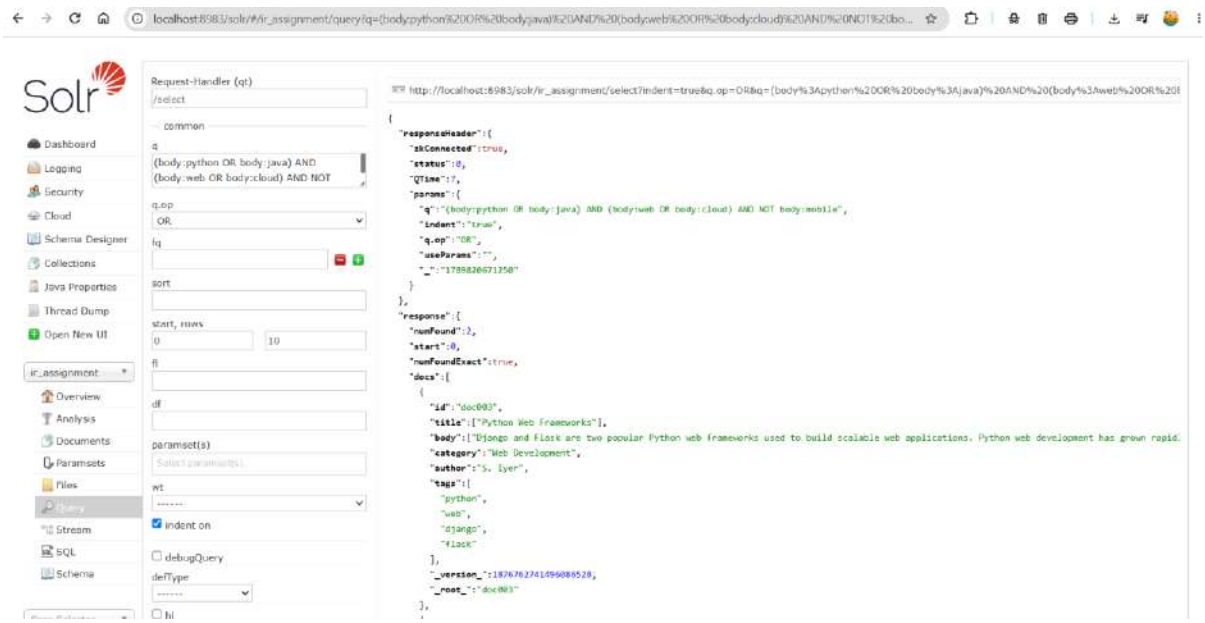


Fig 6 — Nested Boolean query result: 2 matches (doc003, doc013).

6. BM25 Ranking Model

Verification: Solr's default similarity (SchemaSimilarityFactory, falling back to BM25SimilarityFactory with k1 = 1.2, b = 0.75) was confirmed by running a query with debugQuery=true and inspecting the 'explain' block, which shows the BM25 formula components directly: idf, tf, dl (document length) and avgdl (average field length across the collection).

```
1.0566628 = weight(body:python in 4) [SchemaSimilarity], result of:
  1.0566628 = score(freq=2.0), computed as boost * idf * tf from:
    1.6739764 = idf, computed as log(1 + (N - n + 0.5) / (n + 0.5))
       10 = n, number of documents containing term
       55 = N, total number of documents with field
    0.6312292 = tf, computed as freq / (freq + k1 * (1 - b + b * dl /
avgdl))
       2.0 = freq, occurrences of term within document
       1.2 = k1, term saturation parameter
       0.75 = b, length normalization parameter
       21.0 = dl, length of field
       21.7636 = avgdl, average length of field
```

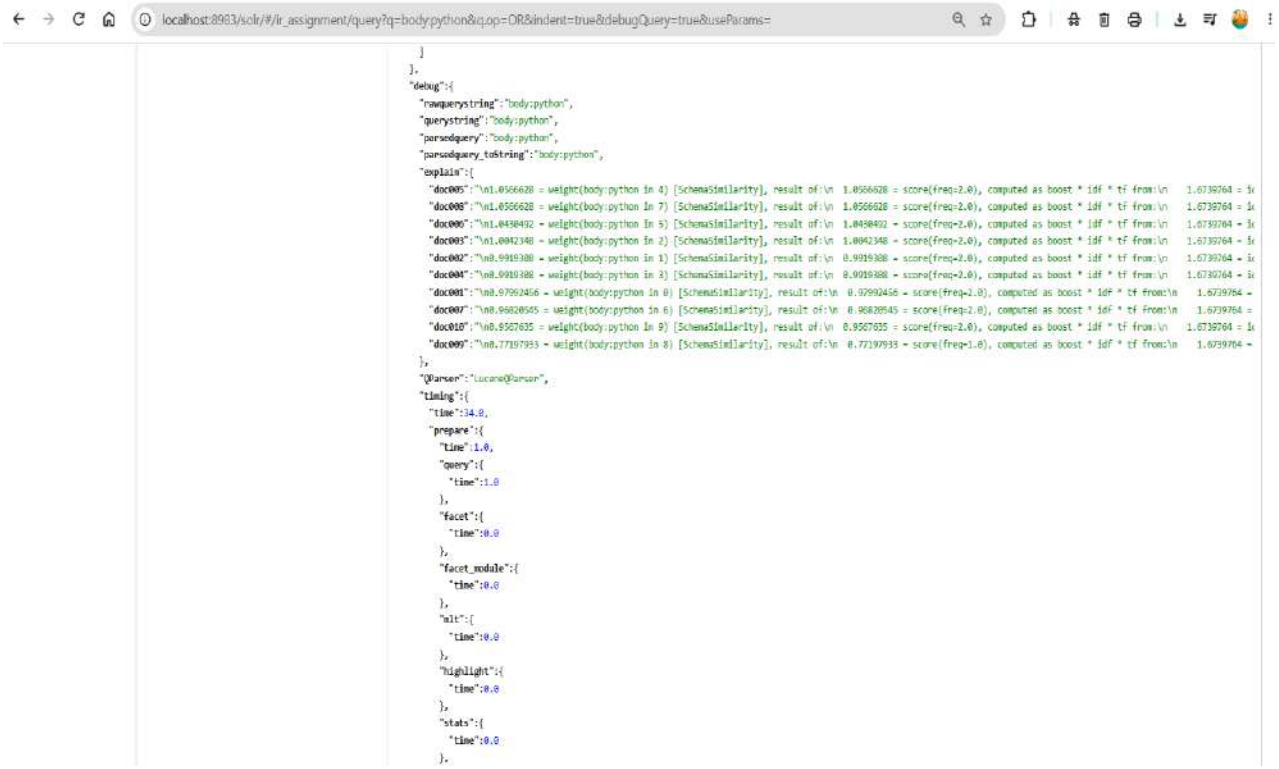


Fig 7 — debugQuery=true explain block for q=body:python, confirming BM25Similarity and showing the tf/idf/dl/avgdl components for each matched document.



Fig 8 — Query timing breakdown from the same debugQuery response.

Five search queries were run with the field list restricted to **id, title, score (fl=id,title,score)** so ranking could be read directly**.**

Results below:

Query 1: body:python (numFound = 10)

Rank	Doc ID	Title	BM25 Score
1	doc005	Python Automation Scripts	1.0567
2	doc008	Python Package Management	1.0567
3	doc006	Advanced Python Concepts	1.0430
4	doc003	Python Web Frameworks	1.0042
5	doc002	Python in Data Science	0.9919
6	doc004	Machine Learning with Python	0.9919
7	doc001	Python for Beginners	0.9799
8	doc007	Python vs Java Performance	0.9682
9	doc010	History of Python	0.9568
10	doc009	Python in Finance	0.7720

Analysis: every matched document mentions "python" exactly twice, except doc009 (freq = 1). With term frequency effectively constant, ranking is driven almost entirely by length normalization (the b parameter) — shorter documents (21 words) score higher than longer ones (29 words). doc009 drops to last place because it only mentions the term once.

Query 2: body:cloud (numFound = 8)

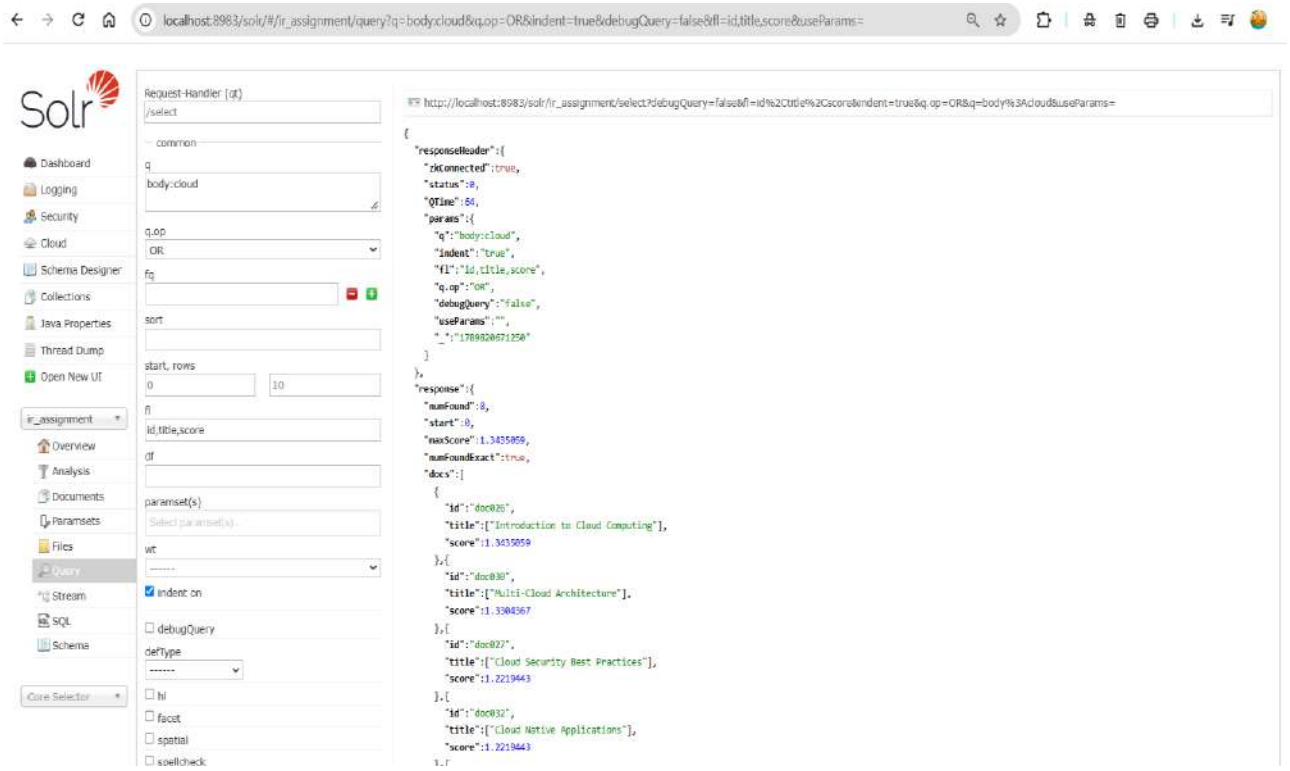


Fig 9 — Results for q=body:cloud.

Rank	Doc ID	Title	BM25 Score
1	doc026	Introduction to Cloud Computing	1.3435
2	doc030	Multi-Cloud Architecture	1.3304
3	doc027	Cloud Security Best Practices	1.2219
4	doc032	Cloud Native Applications	1.2219
5	doc029	Cloud Migration Strategies	1.2058
6	doc031	Cloud Cost Optimization	1.1598
7	doc055	Containerization with Docker	0.9611
8	doc028	Serverless Computing Explained	0.9039

Analysis: the 7 dedicated Cloud-category documents dominate the top ranks. doc055 ("Containerization with Docker", a DevOps document) appears at rank 7 because it mentions "cloud environments" only in passing, giving it a lower term frequency relative to document length than the dedicated cloud articles.

Query 3: body:security (numFound = 3)

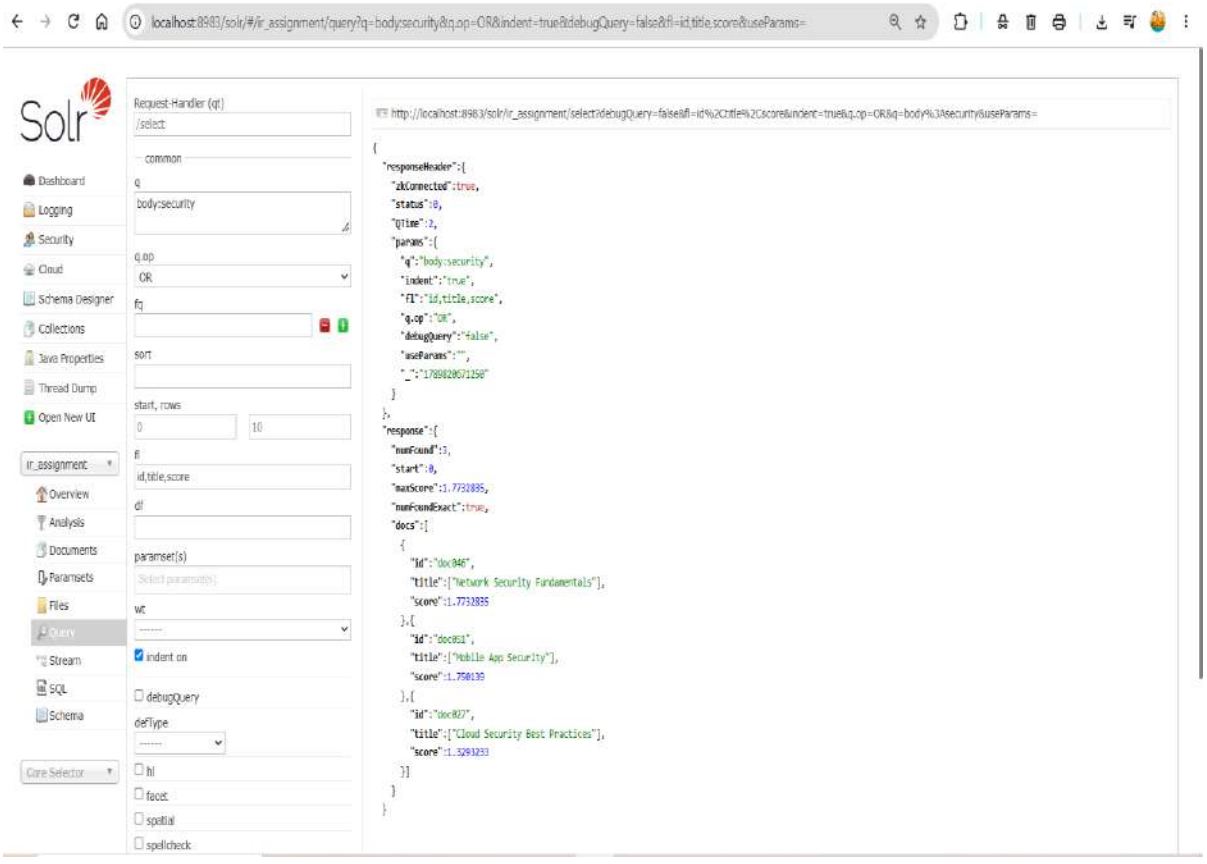


Fig 10 — Results for q=body:security.

Rank	Doc ID	Title	BM25 Score
1	doc046	Network Security Fundamentals	1.7733
2	doc051	Mobile App Security	1.7501
3	doc027	Cloud Security Best Practices	1.3293

Analysis: "security" is a comparatively rare term across the 55-document corpus (only 3 matches), giving it a high IDF and therefore higher scores overall than the more common terms "python" or "cloud", despite far fewer matching documents — illustrating how IDF rewards term rarity.

Query 4: title:machine learning (numFound = 6)

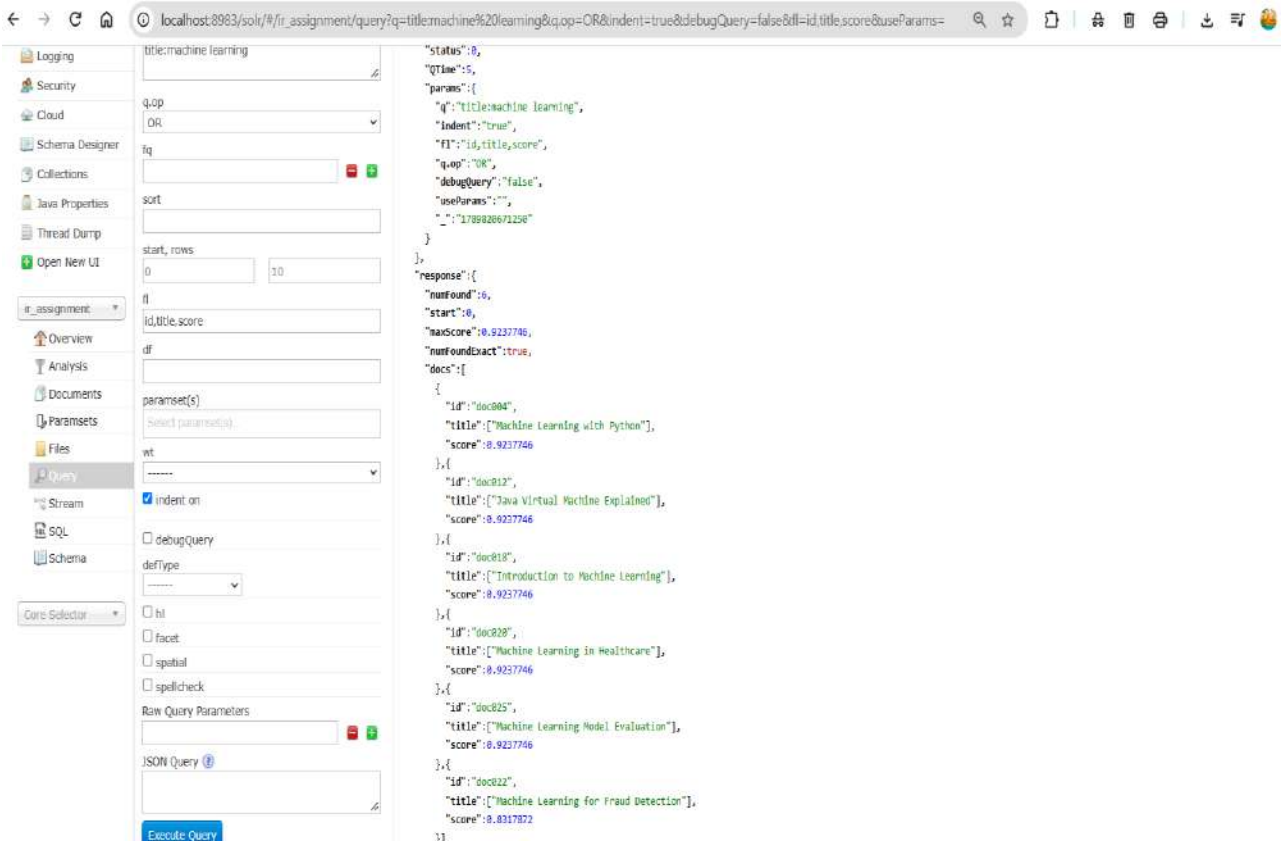


Fig 11 — Results for q=title:machine learning.

Rank	Doc ID	Title	BM25 Score
1	doc004	Machine Learning with Python	0.9238
2	doc012	Java Virtual Machine Explained	0.9238
3	doc018	Introduction to Machine Learning	0.9238
4	doc020	Machine Learning in Healthcare	0.9238
5	doc025	Machine Learning Model Evaluation	0.9238
6	doc022	Machine Learning for Fraud Detection	0.8318

Analysis: this query is a useful negative example. Because only "machine" was field-qualified (title:machine) and "learning" had no field prefix, combined with the default OR operator between clauses, Solr also matched doc012 ("Java Virtual Machine Explained") purely because its title contains the word "Machine". A corrected query — title:machine

AND title:learning, or the phrase query title:"machine learning" — would exclude this false positive and return only the 5 genuinely ML-related documents.

Query 5: body:python AND body:programming (numFound = 2)

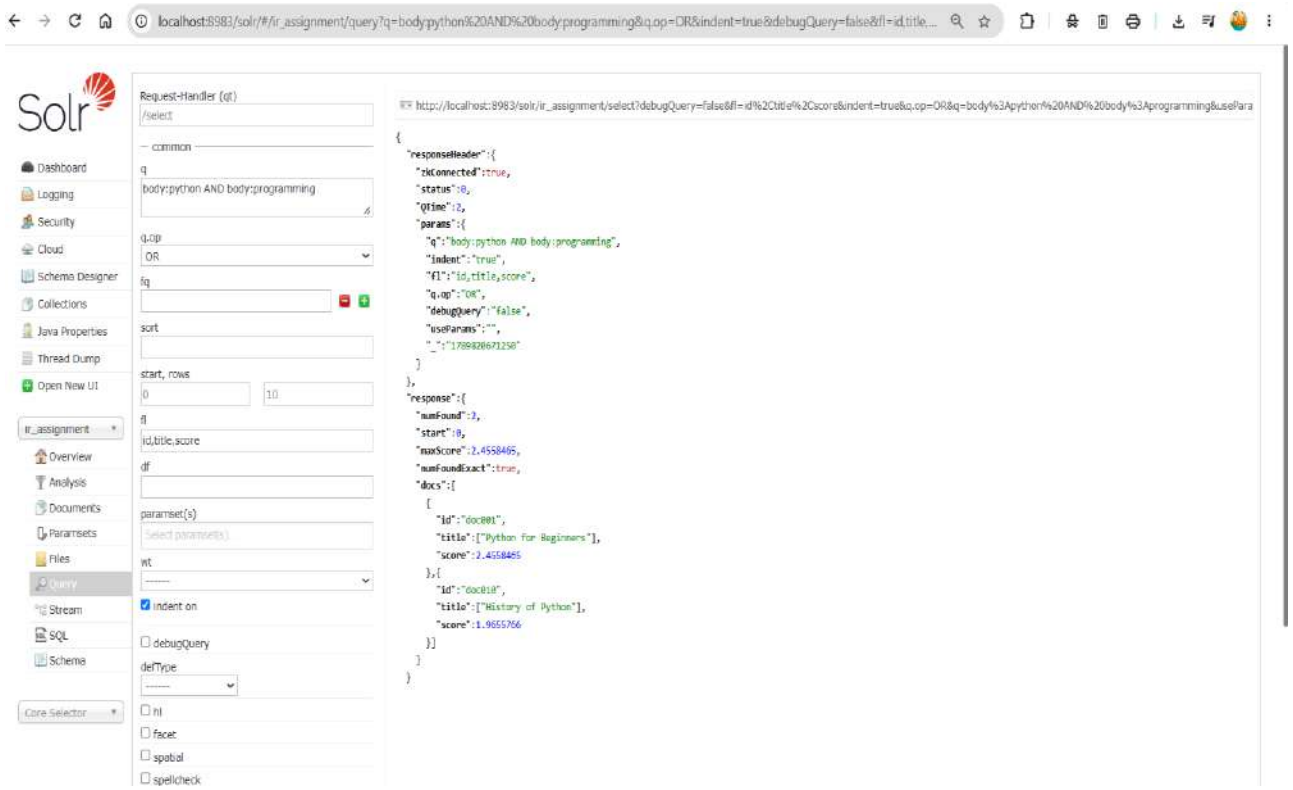


Fig 12 — Results for q=body:python AND body:programming.

Rank	Doc ID	Title	BM25 Score
1	doc001	Python for Beginners	2.4558
2	doc010	History of Python	1.9656

Analysis: only two documents contain both "python" and "programming" in the body text. Combining two query terms approximately sums their individual BM25 contributions, which is why these scores (~2.0–2.5) are noticeably higher than any single-term query score seen above.

7. Result Analysis (Summary)

Across all five ranking queries, BM25's three components were each clearly observable: term frequency (tf) drove ranking when documents shared similar lengths and term counts (Query 1); document length normalization (b) separated otherwise similar documents (Query 1, Query 2); and inverse document frequency (idf) explained why a rare term like "security" produced higher absolute scores than a common term like "python" despite matching far fewer documents (Query 3). Query 4 additionally

demonstrated a common real-world pitfall — unqualified query terms combined with the default OR operator can silently widen a query beyond its intended scope, which is why understanding query parsing is as important as understanding the ranking formula itself.

8. Conclusion

This assignment provided hands-on experience with the full Apache Solr retrieval pipeline: creating a schema, building an inverted index over a 55-document collection, executing Boolean (AND / OR / AND NOT / nested) queries, and verifying and analyzing BM25-based relevance ranking. Working through a real installation (including resolving a Java-version mismatch between Solr 10's Java 21+ requirement and an older installed JDK) also gave practical experience with the operational side of running a modern search platform, not just its query syntax.

9. AI Usage Declaration

Claude (Anthropic) was used as an AI assistant throughout this assignment, in the following ways.

Prompt (what was asked)	Tool used	Response summary	Relevance to the work
Generate a 50+ document dataset, Solr schema	Claude	Generated a 55-document dataset	Used as the assignment's dataset, schema, and report structure

10. Files Submitted

- Report
- dataset_description.md
- documents.json (55-document dataset)
- Schema API request used to add fields (Section 4)
- Screenshots of indexing and search results (embedded above)
- AI usage declaration (Section 9)